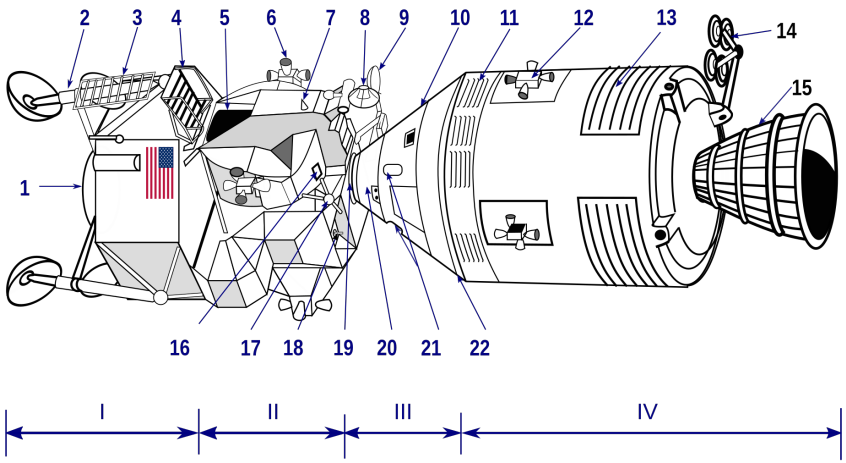
Welcome to
ASEN 3502
Computational Methods

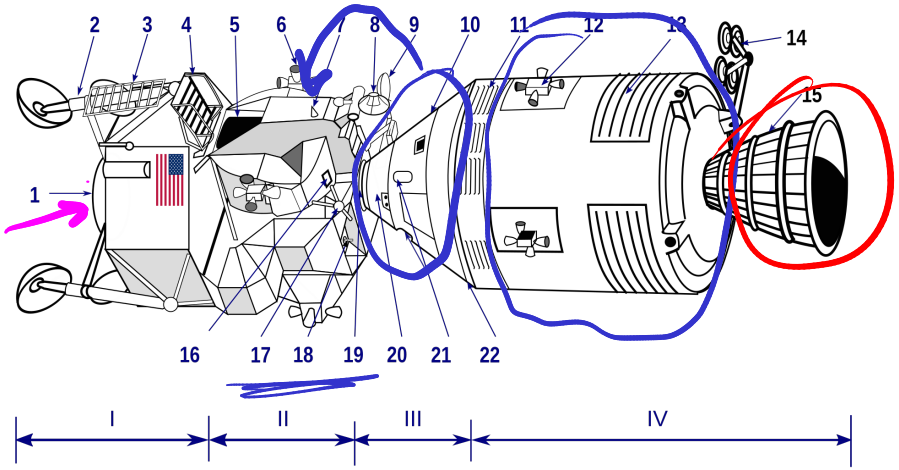
Apollo 13



I–II: lunar module (descent, ascent stages); III: command module; IV: service module.

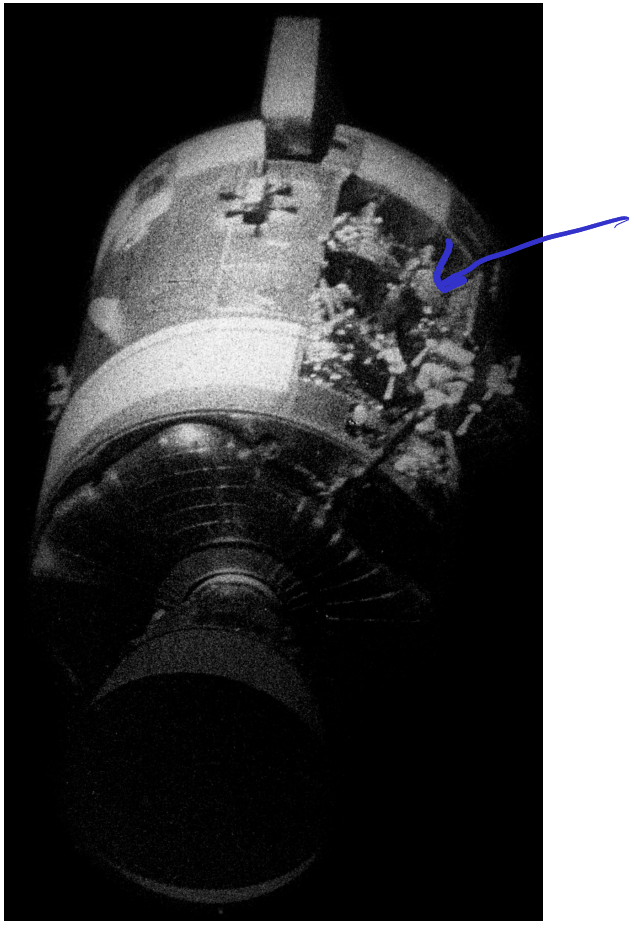
Popadius, Wikimedia Commons, CC0.

Apollo 13



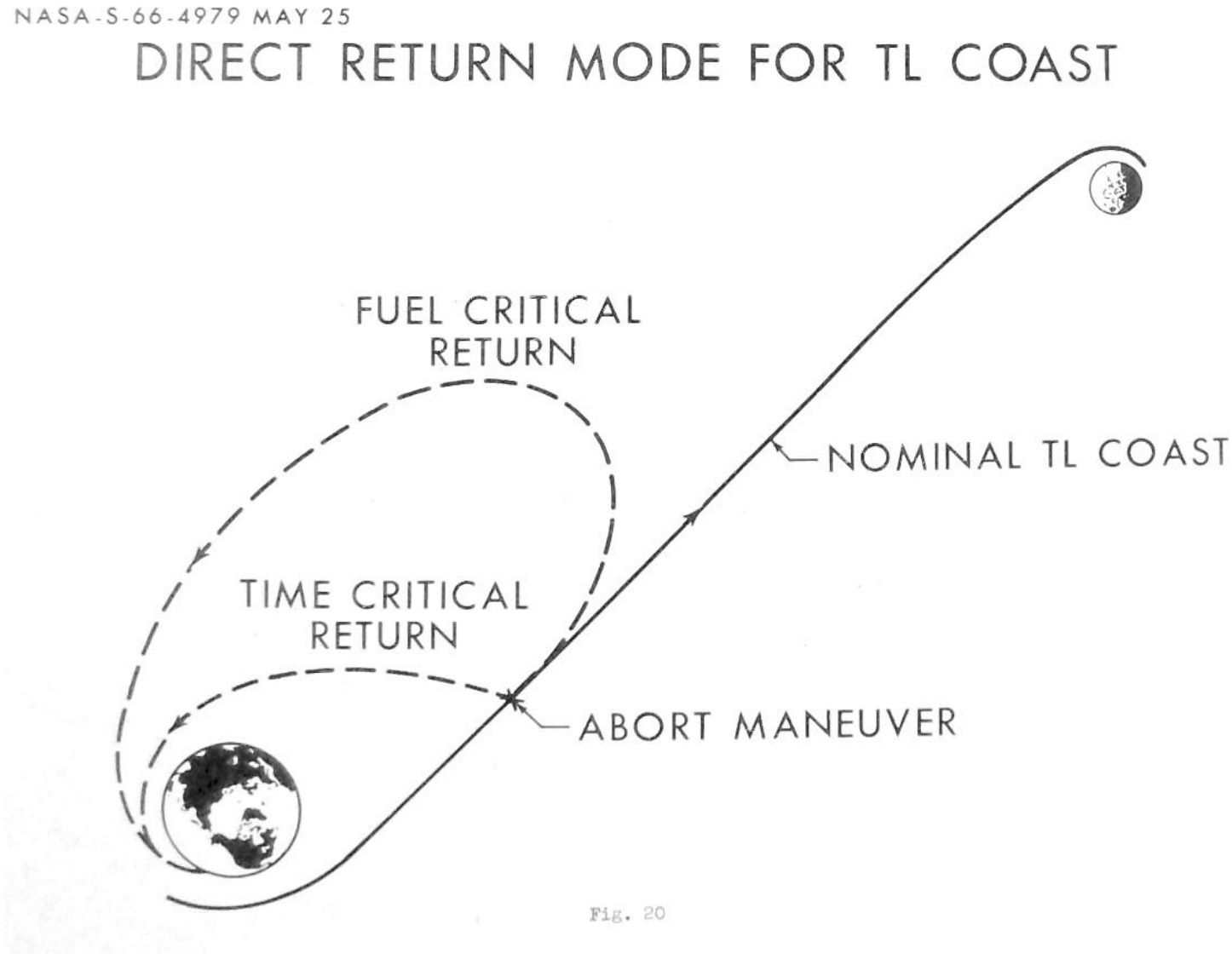
I–II: lunar module (descent, ascent stages); III: command module; IV: service module.

Popadius, Wikimedia Commons, CC0.

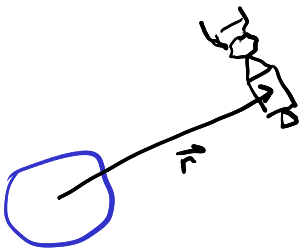


The service module after separation, with one panel blown off. NASA (scan by Kipp Teague), public domain.

Direct return

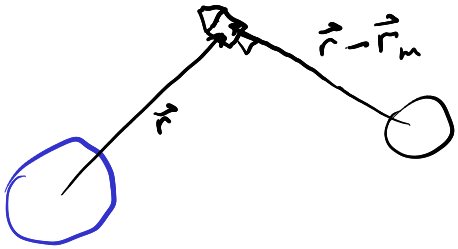


R. L. Berry, *Apollo Lunar Landing Mission Symposium*, NASA MSC, June 1966. Public domain, via Wikimedia Commons.



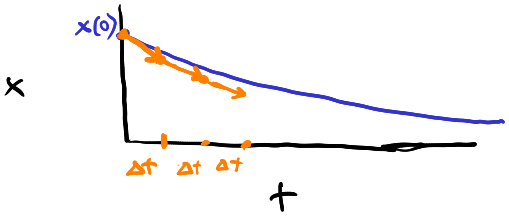
$$\ddot{\vec{r}} = -\frac{\mu_e}{r^2}\frac{\vec{r}}{r} \rightarrow \text{elliptical hyperbolic}$$

$$\mu_e = G m_e$$



$$\ddot{\vec{r}} = -\frac{\mu_e}{r^2}\frac{\vec{r}}{r} - \frac{\mu_m}{\|\vec{r}-\vec{r}_m\|^2}\frac{\vec{r}-\vec{r}_m}{\|\vec{r}-\vec{r}_m\|}$$

no easy analytical solution



$$\dot{x} = -ax$$

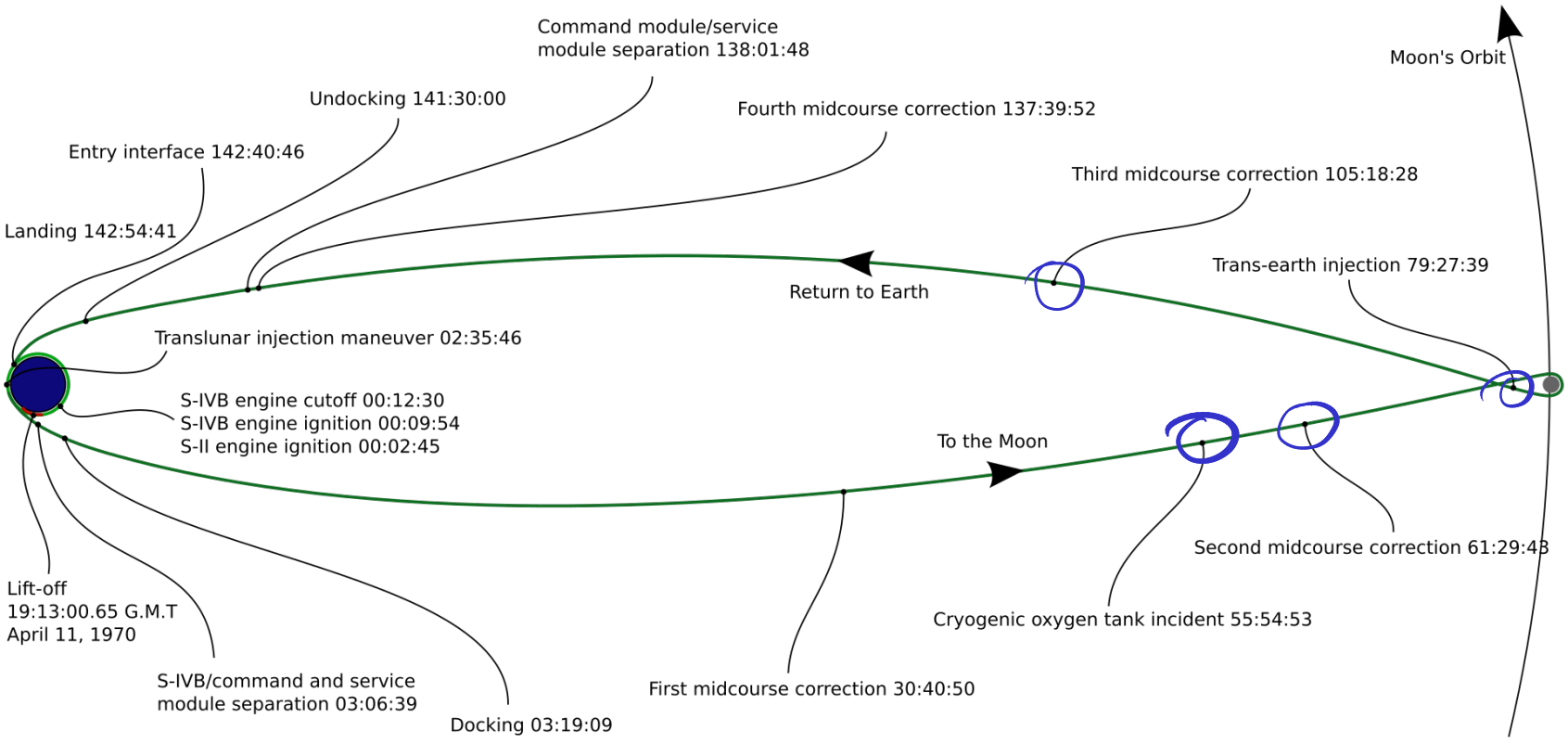
Euler's Method

Error = $O(\Delta t)$

Runge-Kutta

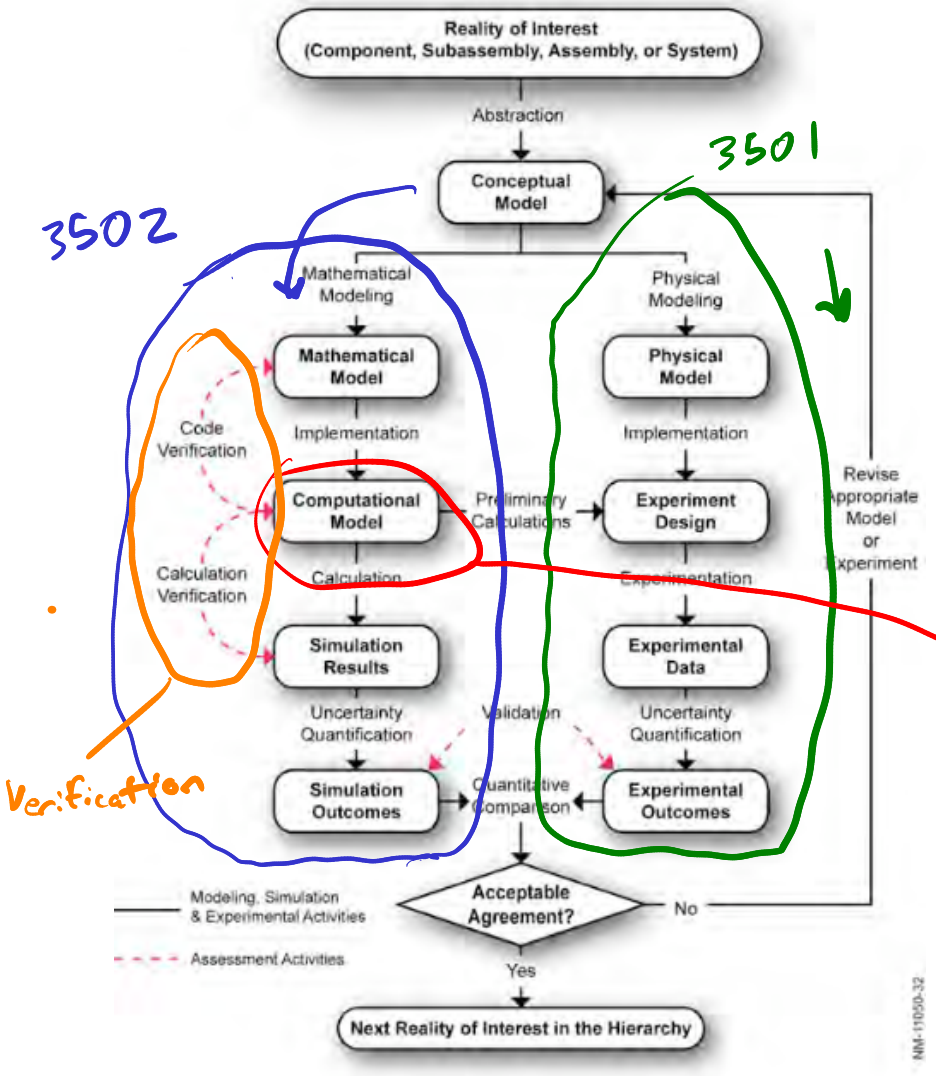
Error = $O(\Delta t^4)$

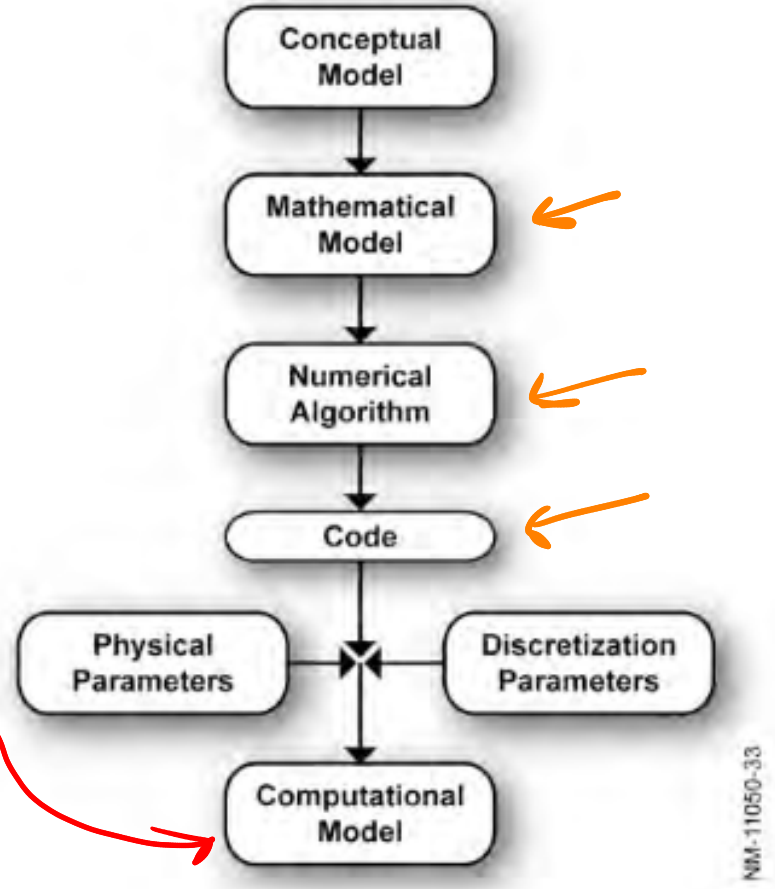
What Apollo 13 actually flew



AndrewBuck, Wikimedia Commons, CC BY-SA 4.0. Source for figures: Apollo 13 Mission Report, p. 3-2.

ASME V&V 10





Your code is not done until you verify it with tests

ASME V&V 10, *Guide for Verification and Validation in Computational Solid Mechanics.*

asen3502.com

Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn
- It may take multiple ways of explaining a concept before a student understands





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn
- It may take multiple ways of explaining a concept before a student understands
- We must work together to help you find the way to understand





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn
- It may take multiple ways of explaining a concept before a student understands
- We must work together to help you find the way to understand
- Taking notes in class and doing the homework yourselves are critical steps to understanding





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn
- It may take multiple ways of explaining a concept before a student understands
- We must work together to help you find the way to understand
- Taking notes in class and doing the homework yourselves are critical steps to understanding
- I am trying to bring you to the point of understanding, not just give you the answer





Our Assumptions

- The material is challenging, and can take multiple engagements to understand fully
- A student who comes to my office or asks a question wants to learn
- It may take multiple ways of explaining a concept before a student understands
- We must work together to help you find the way to understand
- Taking notes in class and doing the homework yourselves are critical steps to understanding
- I am trying to bring you to the point of understanding, not just give you the answer
- We will have fun this semester





8/26

Today

- Linear Systems of Equations
- Gaussian Elim.
- Partial Pivoting

$$2x_1 + x_2 = 4$$
$$-x_1 + x_2 = 1$$

Unkowns: $x_1, \dots x_n$

Linear Eqn.: $\sum_{j=1}^{n} a_j x_j = b$

no nonlinear functions of x_i: x^2, e^x, $\log(x)$, $\sin(x)$ etc.

System of m linear equations $\sum_{j=1}^{n} a_{ij} x_j = b_i \quad i = 1 \dots m$

$$\begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \\ a_{m1} & & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

Book: $[A]\{x\} = \{b\}$

Handwriting: $A\vec{x} = \vec{b}$

for now $m = n$

HW0 $\Rightarrow A \cdot x$

Lab $Ax = b$ (known, known, unknowns)

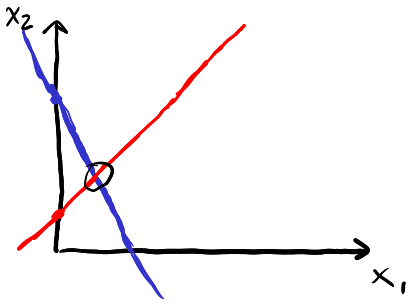
$2x_1 + x_2 = 4$

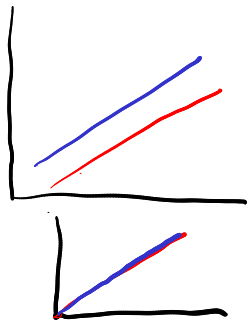
$x_2 = -2x_1 + 4$

$-x_1 + x_2 = 1$

$x_2 = x_1 + 1$

$x_1 = 1, \; x_2 = 2$





Example:

$$\begin{bmatrix} 3 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \end{bmatrix}$$

$3x_1 - x_2 = 0 \rightarrow 3x_1 - 3 = 0$

$x_1 = 1$

$2x_2 = 6$

$x_2 = 3$

$$\begin{bmatrix} 1 & & & & \\ 0 & 1 & & & \\ \vdots & \ddots & \ddots & & \\ 0 & \cdots & 0 & 1 & \\ 0 & \cdots & \cdots & 0 & 1 \end{bmatrix}$$

$$2x_1 + 3x_2 + 4x_3 = 19 \quad (1)$$
$$4x_1 + 11x_2 + 14x_3 = 55 \quad (2)$$
$$2x_1 + 8x_2 + 17x_3 = 50 \quad (3)$$

$$\left[\begin{array}{ccc|c} 2 & 3 & 4 & 19 \\ 4 & 11 & 14 & 55 \\ 2 & 8 & 17 & 50 \end{array}\right]$$

f

$-2 \cdot (1)$

$-1 \cdot (1)$

$$\left[\begin{array}{ccc|c} 2 & 3 & 4 & 19 \\ 0 & 5 & 6 & 17 \\ 0 & 5 & 13 & 31 \end{array}\right]$$

$-1 \cdot (2)$

$$\left[\begin{array}{ccc|c} 2x_1 + 3x_2 + 4x_3 & = & 19 \\ 0 & 5x_2 + 6x_3 & = & 17 \\ 0 & 0 & 7x_3 & = & 14 \end{array}\right]$$

$x_3 = 2$

$2x_1 + 3 \cdot 1 + 4 \cdot 2 = 19$

$2x_1 = 8$

$x_1 = 4$

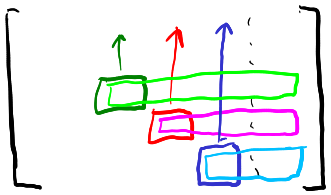
$5x_2 + 12 = 17$

$x_2 = 1$



Eliminate ▭ with ▭
Eliminate ▭ with ▭
etc

Back substitution



Solve for ▭ with ▭
Solve for ▭ with ▭
etc.

Example:

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 3 & 9 & -2 \\ 2 & 1 & -1 \end{bmatrix} \qquad b = \begin{bmatrix} 1 \\ 9 \\ 2 \end{bmatrix}$$

$$x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$$

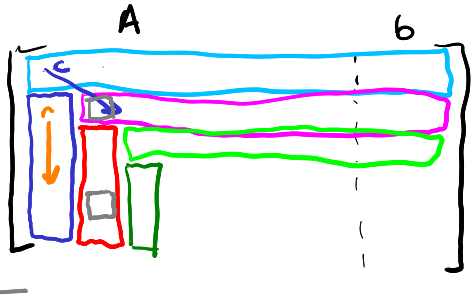
input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$
output: solution $\{x\}$ of $[A]\{x\} = \{b\}$

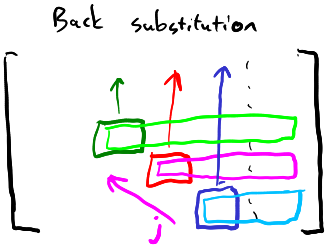
Elimination

```
// forward elimination
for c ← 1 to n − 1 do
    for r ← c + 1 to n do
        f ← A[r, c] / A[c, c]
        A[r, c:n] ← A[r, c:n] − f · A[c, c:n]
        b[r] ← b[r] − f · b[c]
    end
end
```

Backsub

```
// back substitution
for r ← n, n − 1, . . . , 1 do
    s ← b[r]
    for j ← r + 1 to n do
        s ← s − A[r, j] · x[j]
    end
    x[r] ← s / A[r, r]
end
```





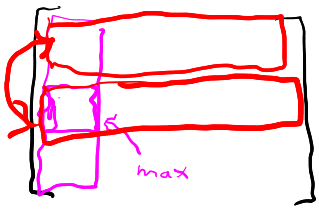
$$A = \begin{bmatrix} 0 & 2 & 3 \\ 4 & 6 & 7 \\ 2 & -3 & 6 \end{bmatrix} \qquad b = \begin{bmatrix} 8 \\ -3 \\ 5 \end{bmatrix}$$

$f \leftarrow Aug[1,2] / Aug[1,1]$ Divide by Zero

Solution: Partial Pivoting

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$
output: solution $\{x\}$ of $[A]\{x\} = \{b\}$

```
// forward elimination
for c ← 1 to n − 1 do
    // find the largest entry in column c
    p ← argmax(abs(A[c:n, c])) + c − 1
    // swap that row into the pivot position
    A[c, :] ↔ A[p, :]
    b[c] ↔ b[p]
    for r ← c + 1 to n do
        f ← A[r, c] / A[c, c]
        A[r, c:n] ← A[r, c:n] − f · A[c, c:n]
        b[r] ← b[r] − f · b[c]
    end
end
// back substitution
for r ← n, n − 1, . . . , 1 do
    s ← b[r]
    for j ← r + 1 to n do
        s ← s − A[r, j] · x[j]
    end
    x[r] ← s / A[r, r]
end
```



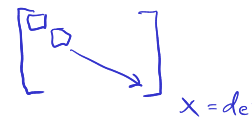
Algorithms and Operation Counting

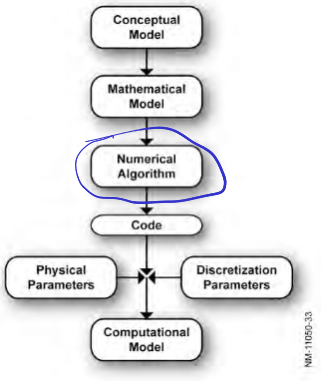
ASEN 3502 — Computational Methods

Please grab a handout in the back!

HW 1 - Due Thursday

Lab min through 1.3
rec through 1.5





Today
- Algorithms
- Operation Counting
- Asymptotic Notation

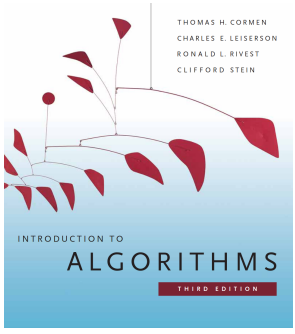
$O(n)$

ASME V&V 10, *Guide for Verification and Validation in Computational Solid Mechanics*.

What is an algorithm?

"Informally, an algorithm is any well-defined computational **procedure** that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**."

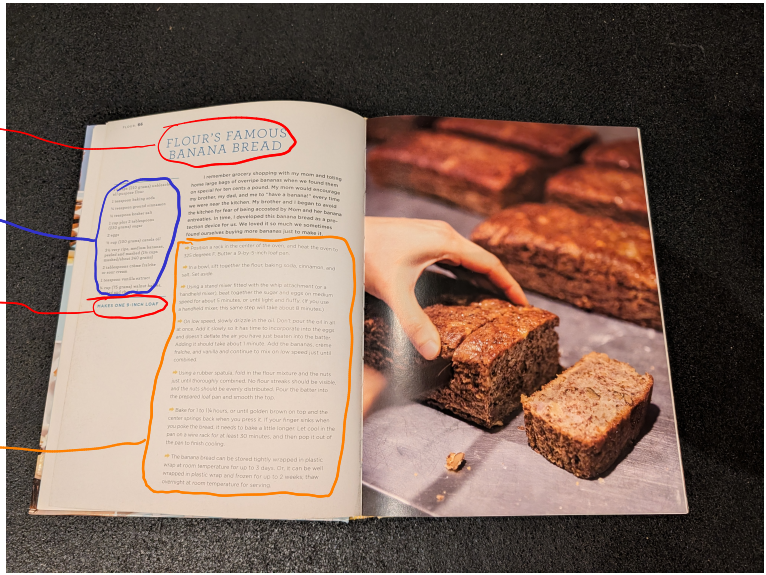
— Cormen et al.



Introduction to Algorithms
Cormen, Leiserson,
Rivest, and Stein

mitpress.mit.edu

Another type of algorithm



Why algorithms?



- Algorithms are mathematical objects. Why are they useful?

Why algorithms?

- Algorithms are mathematical objects. Why are they useful?
  - We can reason about properties like correctness and resource use

Why algorithms?

- Algorithms are mathematical objects. Why are they useful?
  - We can reason about properties like correctness and resource use

    > "An algorithm is said to be **correct** if, for every input instance, it halts with the correct output." — Cormen et al.

Why algorithms?

- Algorithms are mathematical objects. Why are they useful?
  - We can reason about properties like correctness and resource use

    > "An algorithm is said to be **correct** if, for every input instance, it halts with the correct output." — Cormen et al.
  - We can make precise and rigorous statements without having to deal with issues like roundoff error, memory allocation, etc.

How do we represent algorithms?

Matrix-vector multiplication: three ways

English

To get entry r of $\{b\}$, multiply each entry in row r of $[A]$ by the matching entry of $\{x\}$, and add up the results. Do this for every row.

Code

```
b = zeros(n,1);
for r = 1:n
    for c = 1:n
        b(r) = b(r) ...
             + A(r,c)*x(c);
    end
end
```

Pseudocode

input : $n \times n$ matrix $[A]$,
vector $\{x\}$
output : $\{b\} = [A]\{x\}$

for $r \leftarrow 1$ **to** n **do**
  $b[r] \leftarrow 0$
  for $c \leftarrow 1$ **to** n **do**
    $b[r] \leftarrow b[r] + A[r, c] \cdot x[c]$
  end
end

"What separates pseudocode from "real" code is that in pseudocode, we employ whatever expressive method is most clear and concise to specify a given algorithm." — Cormen et al.

Analysis of algorithms

- *Analyzing* an algorithm means predicting the resources that an algorithm requires
  - For today, we will count floating point operations ("Flops"): multiplication, division, addition, and subtraction of floating point numbers

Analysis of algorithms

- *Analyzing* an algorithm means predicting the resources that an algorithm requires
  - For today, we will count floating point operations ("Flops"): multiplication, division, addition, and subtraction of floating point numbers

- **Discuss with your neighbor:** Consider the equation $[A]\{x\} = \{b\}$. As n gets larger, which requires more floating point operations? Finding $\{x\}$ given $[A]$ and $\{b\}$, or finding $\{b\}$ given $[A]$ and $\{x\}$?

① $b = Ax$

② $Ax = b$ Harder if using Gaussian Elimination

Matrix-vector multiplication

input : $n \times n$ matrix $[A]$, vector $\{x\}$
output : $\{b\} = [A]\{x\}$

		flops	times
1	**for** $r \leftarrow 1$ **to** n **do**	0	$\sum_{r=1}^{n} 1 = n$
2	$b[r] \leftarrow 0$	0	n
3	**for** $c \leftarrow 1$ **to** n **do**	0	n^2
4	$b[r] \leftarrow b[r] + A[r,c] \cdot x[c]$	2	n^2
5	**end**		
6	**end**		

$$\sum_{r=1}^{n} \sum_{c=1}^{n} 1 = \sum_{r=1}^{n} n = n^2$$

FLOPS: $2n^2$

Gaussian elimination

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$
output : solution $\{x\}$ of $[A]\{x\} = \{b\}$

```
   // forward elimination
1  for c ← 1 to n − 1 do
2  |   for r ← c + 1 to n do
3  |   |   f ← A[r, c] / A[c, c]
4  |   |   for j ← c to n do
5  |   |   |   A[r, j] ← A[r, j] − f · A[c, j]
6  |   |   end
7  |   |   b[r] ← b[r] − f · b[c]
8  |   end
9  end

   // back substitution
10 for r ← n, n − 1, . . . , 1 do
11 |   s ← b[r]
12 |   for j ← r + 1 to n do
13 |   |   s ← s − A[r, j] · x[j]
14 |   end
15 |   x[r] ← s / A[r, r]
16 end
```

	flops	times
	0	$\sum_{i=1}^{n-1} 1 = n-1$
→	1	$\frac{n^2-n}{2}$
	0	
→	2	$\frac{n^3}{3} + O(n^2)$
→	2	$\frac{n^2-n}{2}$
	0	n
	0	n
	0	$\frac{n^2-n}{2}$
→	2	
→	1	n

Useful sum identities

$$\sum_{i=1}^{m} c f(i) = c \sum_{i=1}^{m} f(i) \qquad \sum_{i=1}^{m} f(i) + g(i) = \sum_{i=1}^{m} f(i) + \sum_{i=1}^{m} g(i)$$

$$\sum_{i=1}^{m} 1 = m \qquad \sum_{i=k}^{m} 1 = m - k + 1$$

$$\sum_{i=1}^{m} i = \frac{m(m+1)}{2} \qquad \sum_{i=1}^{m} i^2 = \frac{m(m+1)(2m+1)}{6} = \frac{m^3}{3} + O(m^2)$$

$$\sum_{c=1}^{n-1} \sum_{r=c+1}^{n} 1 = \sum_{c=1}^{n-1} n - c = \sum_{c=1}^{n-1} n - \sum_{c=1}^{n-1} c$$

$$= n(n-1) - \frac{(n-1)n}{2} = \frac{n^2-n}{2}$$

$$\sum_{c=1}^{n-1} \sum_{r=c+1}^{n} \sum_{j=c}^{n} 1 = \frac{n^3}{3} - \frac{n}{3}$$

see flop-count.pdf section 2.2

https://asen3502.com/lectures/020-algorithms/flop-count.pdf

FLOPS : $\frac{n^2-n}{2} + 2\left(\frac{n^3}{3} + O(n^2)\right) + 2\frac{n^2-n}{2}$

FLOPS $2\frac{n^3}{3} + O(n^2) + 2\frac{n^2-n}{2} + n$

```
function [C] = matmult([A], [B])
    for i = 1 to n
        for j = 1 to n
            c_ij = 0
            for k = 1 to n
                c_ij = c_ij + a_ik b_kj
            end
        end
    end
    return [C]
end
```

flops	times
0	n
0	n^2
0	⋮
0	n^3
2	n^3

Asymptotic notation

- Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote

Asymptotic notation

- Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote
- Chapra: $O(m^n)$ means "terms of order m^n and lower"

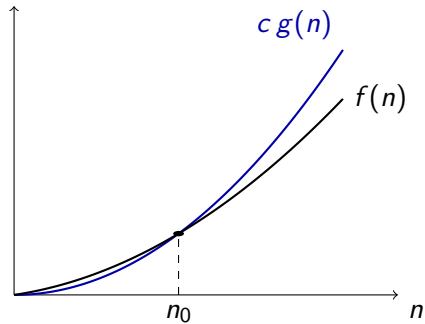
Asymptotic notation

- Asymptotic notation is a tool for keeping track of only the terms that are most important as a function approaches an asymptote
- Chapra: $O(m^n)$ means "terms of order m^n and lower"

Definition

$f(n) = O(g(n))$ as $n \to \infty$ if there exist positive constants c and n_0 such that

$$0 \leq f(n) \leq c\,g(n) \quad \text{for all } n \geq n_0$$



$f(n) \leq c\,g(n)$ for all $n \geq n_0$

← $f(n) = 3n^2 + 5n + 2$

$f(n) = 3n^2 + 5n + 2$

$c = 4.0$

$n_0 = 6$

then $0 \leq f(n) \leq c\,g(n)$

for all $n \geq n_0$

Therefore $f(n) = O(n^2)$

$c \geq$ sum of coefficients

$n_0 = 1$

$c = 10$

$n_0 = 1$

Useful Rules

1) $c\,f(n) = O(f(n)) \quad \forall\ c > 0$
2) if $f_1(n) = O(n^a)$ and $f_2(n) = O(n^b)$, $f_1(n) + f_2(n) = O(n^{\max(a,b)})$
3) if $f_1(n) = O(n^a)$ and $f_2(n) = O(n^b)$, $f_1(n) \cdot f_2(n) = O(n^{a+b})$
4) if $|f(n)|$ is bounded, $f(n) = O(1)$

Example: $f(n) = n^2 + 12n\sin(n) + 50 = O(n^2)$

n^2: $O(n^2)$; $12n\sin(n)$: $O(n)$ $O(1)$; 50: $50 \cdot 1$, $O(1)$

Gaussian elimination

input : $n \times n$ matrix $[A]$, right-hand side $\{b\}$
output : solution $\{x\}$ of $[A]\{x\} = \{b\}$

```
// forward elimination
for c ← 1 to n − 1 do                                  ≤ n
    for r ← c + 1 to n do                              ≤ n
        f ← A[r, c] / A[c, c]
        for j ← c to n do                              ≤ n
            A[r, j] ← A[r, j] − f · A[c, j]
        end
        b[r] ← b[r] − f · b[c]
    end
end
// back substitution
for r ← n, n − 1, . . . , 1 do
    s ← b[r]
    for j ← r + 1 to n do
        s ← s − A[r, j] · x[j]
    end
    x[r] ← s / A[r, r]
end
```

flops	Order of times called
×	$O(n)$
×	$O(n^2)$
✓	
×	$O(n^3)$
✓	
×	$O(n^2)$
✓	
×	$O(n)$
×	
×	$O(n^2)$
✓	
✓	$O(n)$

$O(n^3)$ Flops

Today

LU - Factorization

HW1 Due tomorrow
HW2 Assigned tonight
Lab 1: catch up with team to
1.3 (minimum)
1.5 (recommended)
before Friday

Study Hall tonight!

Recall: with a triangular matrix, T, solving $Tx = b$ for x is $O(n^2)$

Suppose we have

$$LU = A$$

L: lower triangular, U: upper triangular

$$Ax = b$$
$$L\underbrace{Ux}_{d} = b$$

0. factorize A into LU
1. solve $Ld = b$
2. solve $Ux = d$

both steps solve with a triangular matrix

$$\left(\underbrace{I}_{\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}} + \begin{bmatrix}0&0&0\\2&0&0\\0&0&0\end{bmatrix}\right)\begin{bmatrix}2&3&4\\0&5&6\\2&8&17\end{bmatrix} \overset{\checkmark}{=} \begin{bmatrix}2&3&4\\4&11&14\\2&8&17\end{bmatrix}$$

$\Rightarrow L$ $\Rightarrow U$

$$\left(I + \begin{bmatrix}0&0&0\\2&0&0\\1&0&0\end{bmatrix}\right)\begin{bmatrix}2&3&4\\0&5&6\\0&5&13\end{bmatrix} \overset{\checkmark}{=} \quad 11$$

$$\left(I + \begin{bmatrix}0&0&0\\2&0&\\1&1&0\end{bmatrix}\right)\begin{bmatrix}2&3&4\\0&5&6\\0&0&7\end{bmatrix} \overset{\checkmark}{=} \quad 11$$

$$\underbrace{\begin{bmatrix}1&0&0\\2&1&0\\1&1&1\end{bmatrix}}_{L}\underbrace{\begin{bmatrix}2&3&4\\0&5&6\\0&0&7\end{bmatrix}}_{U} = \begin{bmatrix}2&3&4\\4&11&14\\2&8&17\end{bmatrix}$$

$$L = \begin{bmatrix}1&&&\\f_{21}&1&&\\f_{31}&f_{32}&\ddots&\\\vdots&&&\\f_{n1}&&f_{n,n-1}&1\end{bmatrix}$$

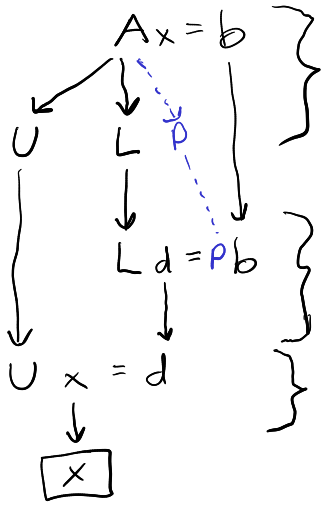
U = result of GE

f_{ij} = factor from GE for eliminating element ij

LU factorization: run GE and save factors in L

$$A = \begin{bmatrix}2&1\\6&5\end{bmatrix}\begin{matrix}①\\②\end{matrix}$$ subtract $3 \times ①$ from ②

$$L = \begin{bmatrix}1&0\\3&1\end{bmatrix} \quad U = \begin{bmatrix}2&1\\0&2\end{bmatrix} \quad LU = \begin{bmatrix}2&1\\6&5\end{bmatrix} \overset{\checkmark}{=} A$$



Factorization $O(n^3)$ only involves A not b!

(Forward) Substitution $O(n^2)$

(Back) Substitution $O(n^2)$

If we have many b's to solve for, factorize once $(O(n^3))$ then do forward and back substitution for each b $(O(n^2))$!

What about Pivoting?

$LU = PA$

↑ permutation matrix (keep track of pivots)

$$\begin{bmatrix} 4 & 11 & 14 \\ 2 & 3 & 4 \\ 2 & 8 & 17 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 & 4 \\ 4 & 11 & 14 \\ 2 & 8 & 17 \end{bmatrix}$$

A' P A

each row i determines which row j of A ends up in A'

$p_{ij} = 1$ if row j of A becomes row i of A'

Exercise 2

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ c_1 & c_2 & c_3 \\ b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

$$\begin{bmatrix} -c- \\ -b- \\ -a- \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

$Ax = b$

$LU = PA$

$PAx = Pb$

$LUx = Pb$

Matrix Inverse
Condition Number
Roundoff Error

Homework 2 Due Thursday

$$Ax = b$$

inverse → $A^{-1}b = x$

$$AA^{-1}b = Ax = b$$

$$AA^{-1}b = b$$

$$AA^{-1} = I \quad (3\times 3)$$

$$A\begin{bmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ a_{31}^{-1} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \qquad A\begin{bmatrix} a_{12}^{-1} \\ a_{22}^{-1} \\ a_{32}^{-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \qquad A\begin{bmatrix} a_{13}^{-1} \\ a_{23}^{-1} \\ a_{33}^{-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Matrix Inverse via LU factorization

input : $n \times n$ matrix $[A]$
output: $[A]^{-1}$

$[L], [U] \leftarrow$ `lu_decompose`$([A])$
for $k \leftarrow 1$ **to** n **do**
  $\{e_k\} \leftarrow$ unit vector with 1 at element k; 0 elsewhere
  $A^{-1}[:, k] \leftarrow$ `lu_solve`$([L], [U], \{e_k\})$
end

What can go wrong? from APPM 2360

$Ax = b$ does not have a unique solution $\iff$ A is singular $\iff$ $\det(A) = 0$ $\iff$ A is not full rank $\iff$ A^{-1} does not exist

$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$ not full rank

What if A is almost singular?

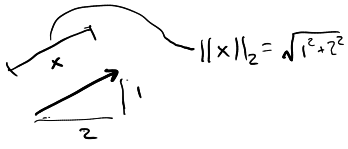
$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

large cond $\Rightarrow$ nearly singular

$\| \ \|$ means "norm" Book covers many types of norms

For now $\| \ \| = \| \ \|_2$ "2-norm"

For a vector x, $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots x_n^2}$, "length" of x



For a matrix A, $\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sqrt{\mu_{max}}$ ← max eigenvalue

"The most A can elongate a vector"

Key property of cond:

$$\frac{\|\Delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\Delta A\|}{\|A\|}$$

relative error in result

relative error in matrix

Roundoff Errors

Computer uses only 3 significant figures

747 takeoff mass

3.97×10^5 kg burns 4.21 kg/s fuel

$$\begin{array}{rl} 397{,}000 & \text{kg} \\ - \quad 4.21 & \text{kg} \\ \hline 396{,}995.71 & \end{array} \longrightarrow \text{3 sig figs} \longrightarrow 3.97 \times 10^5 \text{ kg}$$

How computers store numbers: bits (1 or 0)

Humans think in decimal digits 0, ..., 9

Computers think in Binary 0, 1

Unsigned Integers

	1000	100	10	1
3	0	0	0	3
137	0	1	3	7
43	0	0	4	3

1 byte

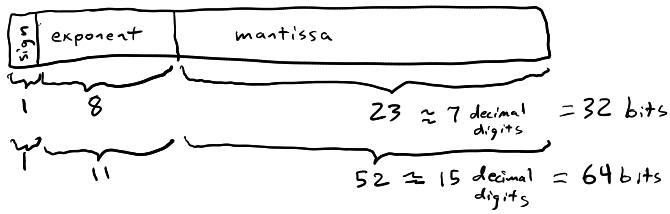
128	64	32	16	8	4	2	1
0	0	0	0	0	0	1	1
1	0	0	0	1	0	0	1
0	0	1	0	1	0	1	1

Scientific Notation

$$\pm s \times 10^{e}$$ (e: exponent; s: significand)

Floating Point IEEE 754

$$\pm (1+f) \times 2^{(e-127)}$$ (e: exponent; f: mantissa)



precision
single
double

matlab/julia eps(x) distance to next floating point number above x

$$\frac{\|\Delta A\|}{\|A\|} \approx \text{eps}(1.0)$$

$$\frac{\|\Delta x\|}{\|x\|} \lesssim \text{cond}(A)\,\varepsilon$$

Please Grab a handout in the back.

Today

Solving Nonlinear Function

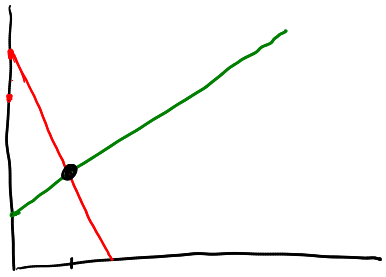
Bracketing Methods: Most important: Bisection

Announcements / Reminders

Lab 1 due Thursday - Check Auto-grader!

HW 1 Released (Graded)

Linear

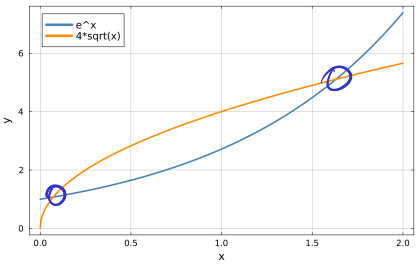


$-2x + 4 = x + 1$

$x_2 = -2x_1 + 4$ $\quad x_2 = x_1 + 1$

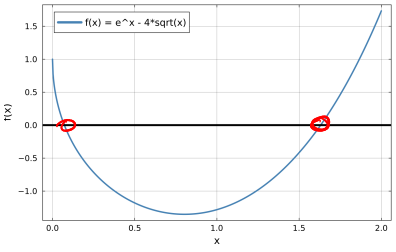
$x = 1$

Nonlinear



$e^x = 4\sqrt{x}$

no analytical in "simple functions"



$f(x) = 0 = e^x - 4\sqrt{x}$

find "roots"

x-values where $f(x) = 0$

Bracketing Open

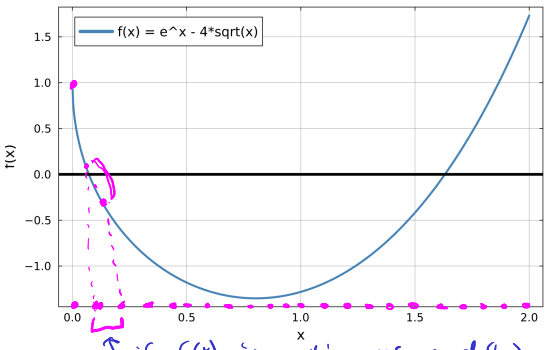
Today

Incremental Search

Incremental Search

input : function f, search range $[x_{\min}, x_{\max}]$, number of steps n
output: list of brackets $[x_l, x_u]$, each containing a root

$\Delta x \leftarrow (x_{\max} - x_{\min}) / n$
brackets $\leftarrow \{\}$
$x_l \leftarrow x_{\min}$
for $k \leftarrow 1$ **to** n **do**
  $x_u \leftarrow x_l + \Delta x$
  if $f(x_l) \cdot f(x_u) < 0$ **then**
    // sign change ⇒ a root lies in $[x_l, x_u]$
    append $[x_l, x_u]$ to brackets
  end
  $x_l \leftarrow x_u$
end
return brackets



if $f(x)$ is continuous and $f(x_l)$ and $f(x_r)$ have different signs, a root is between x_l and x_r

Strength: Can find multiple roots

Weakness: Can miss roots, slow at finding one root (we will see shortly)

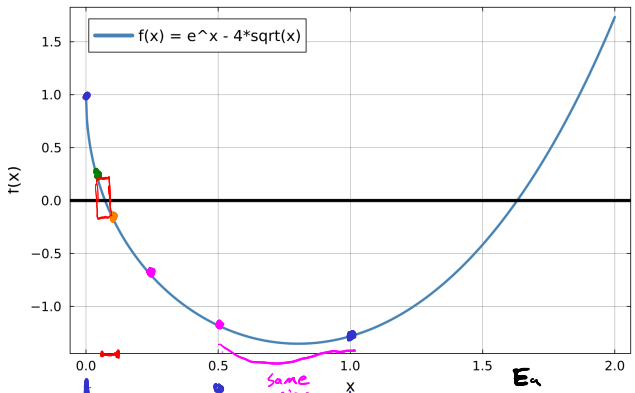
error → $E_a = \frac{\Delta x}{n-1}$

Bisection

Bisection

input : function f, bracket $[x_l, x_u]$ with $f(x_l) \cdot f(x_u) < 0$, tolerance ε_s
output: estimate x_r of the root

$x_r \leftarrow x_l$
repeat
  $x_{r,\text{old}} \leftarrow x_r$
  $x_r \leftarrow (x_l + x_u) / 2$
  // keep the half that still brackets the root
  if $f(x_l) \cdot f(x_r) < 0$ **then**
    $x_u \leftarrow x_r$
  else if $f(x_l) \cdot f(x_r) > 0$ **then**
    $x_l \leftarrow x_r$
  else
    return x_r // exact root
  end
  $\varepsilon_a \leftarrow \left| \frac{x_r - x_{r,\text{old}}}{x_r} \right| \cdot 100\%$
until $\varepsilon_a < \varepsilon_s$
return x_r



Iter:	1		2		3		4		5
	1		1		1		1		H
		•		•		•	1		↑↑↑
		1		1		1			0.0675 0.125

E_a

$\Delta x = 1$

$\frac{\Delta x}{2} = 0.5$

$\frac{\Delta x}{4} = 0.25$

$\frac{\Delta x}{8} = 0.125$

$\frac{\Delta x}{16} = 0.0675$

$E_a = \frac{\Delta x}{2^n}$

$$E_a = \frac{\Delta x}{2^n}$$

function calls for desired $\frac{\Delta x}{E_s}$

Incremental

$E_a = E_s = \frac{\Delta x}{n-1}$

$n - 1 = \frac{\Delta x}{E_s}$

$$n = \frac{\Delta x}{E_s} + 1$$

$O\left(\frac{\Delta x}{E_s}\right)$

Bisection

$E_a = E_s = \frac{\Delta x}{2^n}$

$2^n = \frac{\Delta x}{E_s}$

$$n = \log_2\left(\frac{\Delta x}{E_s}\right)$$

$O\left(\log\left(\frac{\Delta x}{E_s}\right)\right)$

$E_s = 0.01\,\Delta x$

$\frac{\Delta x}{E_s} = 100$

101 func evals

$\log(100) \approx 6.64$

7 function evals !

$E_s = 0.0001\,\Delta x$

$\frac{\Delta x}{E_s} = 10{,}000$

10,001 func evals

14 func evals

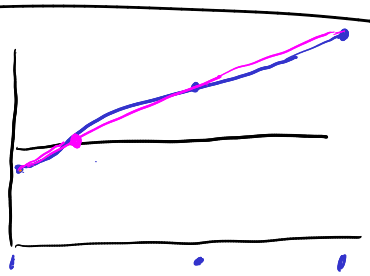
False Position

False Position

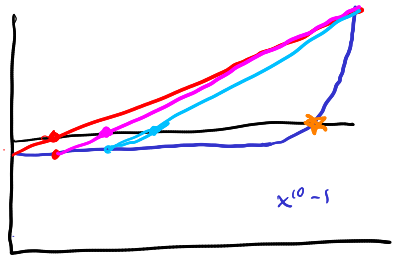
input : function f, bracket $[x_l, x_u]$ with $f(x_l) \cdot f(x_u) < 0$, tolerance ε_s
output: estimate x_r of the root

$x_r \leftarrow x_l$
repeat
  $x_{r,\text{old}} \leftarrow x_r$
  $x_r \leftarrow x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$
  // keep the side that still brackets the root
  if $f(x_l) \cdot f(x_r) < 0$ **then**
    $x_u \leftarrow x_r$
  else if $f(x_l) \cdot f(x_r) > 0$ **then**
    $x_l \leftarrow x_r$
  else
    return x_r // exact root
  end
  $\varepsilon_a \leftarrow \left|\frac{x_r - x_{r,\text{old}}}{x_r}\right| \cdot 100\%$
until $\varepsilon_a < \varepsilon_s$
return x_r

← only difference from bisection



Problem: Doesn't handle curvature well



Today

Open Methods: Newton-Raphson + Friends

Secant, Brent's, Fixed Point Iter (maybe)

Reminders

Lab 1 due tomorrow

First: Errors (Chapra's notation)

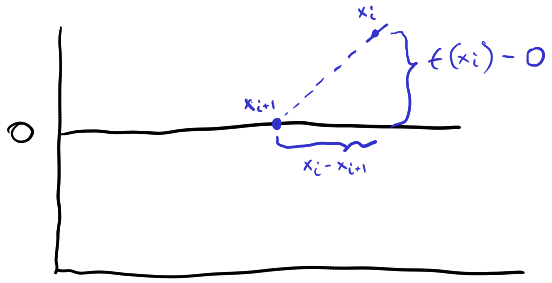
E: absolute error (units)

ε: relative (percent) error

Subscripts

- t: true (typically unknown)
- a: approximate
- s: stopping

Newton-Raphson: use $f'(x)$



$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}} \quad \frac{\text{rise}}{\text{run}}$$

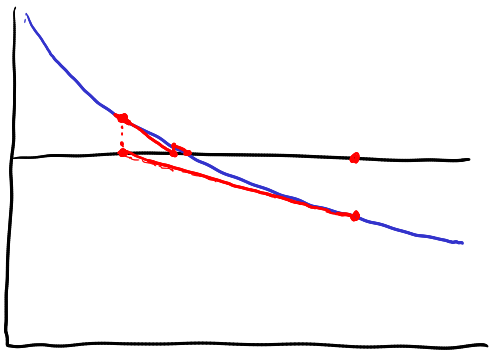
$$(x_i - x_{i+1}) f'(x_i) = f(x_i)$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Newton-Raphson Most widely used

input : function f and its derivative f', initial guess x_0, tolerance ε_s, maximum iterations N
output: estimate x_r of the root

$x_r \leftarrow x_0$
for $i \leftarrow 1$ **to** N **do**
  $x_{r,\text{old}} \leftarrow x_r$
  $x_r \leftarrow x_{r,\text{old}} - \frac{f(x_{r,\text{old}})}{f'(x_{r,\text{old}})}$ ← update
  $\varepsilon_a \leftarrow \left|\frac{x_r - x_{r,\text{old}}}{x_r}\right| \cdot 100\%$
  if $\varepsilon_a < \varepsilon_s$ **then**
    return x_r
  end
end
return x_r // did not converge in N iterations



Beauty of Newton-Raphson: Extremely fast convergence near root!

Bisection

$E_{a,i+1} = \frac{1}{2} E_{a,i}$

"linear convergence"

number of sig figs increases linearly with i

(different terminology from $E_{a,i} = O\left(\left(\frac{1}{2}\right)^k\right)$)

Gauranteed to converge if
- Δx contains root
- $f(x)$ is continuous

fast

Newton Raphson

Near Root ↓ "can be shown"

→ $E_{t,i+1} \simeq -\frac{f''(x_r)}{2f'(x_r)} E_{t,i}^2$

"quadratic convergence"

number of sigfigs doubles every iteration

Can diverge

FAST near root

Downside: Newton requires $f'(x)$

Secant Method

$$f'(x_i) \approx \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

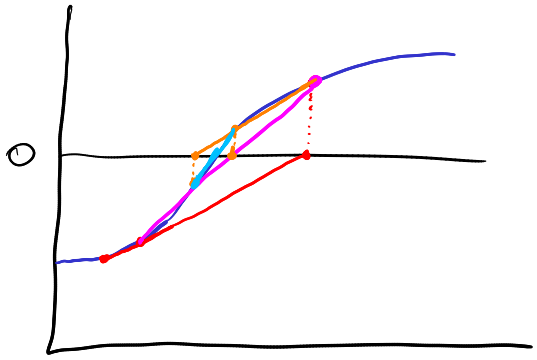
plug into Newton update

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

Secant Method

input : function f, initial guesses x_{-1} and x_0, tolerance ε_s, maximum iterations N
output: estimate x_r of the root

$x_{\text{prev}} \leftarrow x_{-1}$; $x_r \leftarrow x_0$
for $i \leftarrow 1$ **to** N **do**
  $x_{r,\text{old}} \leftarrow x_r$
  $x_r \leftarrow x_{r,\text{old}} - \dfrac{f(x_{r,\text{old}})\,(x_{\text{prev}} - x_{r,\text{old}})}{f(x_{\text{prev}}) - f(x_{r,\text{old}})}$ ← only difference from Newton
  $x_{\text{prev}} \leftarrow x_{r,\text{old}}$
  $\varepsilon_a \leftarrow \left|\dfrac{x_r - x_{r,\text{old}}}{x_r}\right| \cdot 100\%$
  if $\varepsilon_a < \varepsilon_s$ **then**
    return x_r
  end
end
return x_r // did not converge in N iterations



Brent's Method

- Combination of Bracketing and Open Methods
- Does not require derivatives
- Basis for Matlab's fzero

Brent's Method

input : function f, bracket $[a, b]$ with $f(a) \cdot f(b) < 0$, tolerance ε_s
output: estimate b of the root

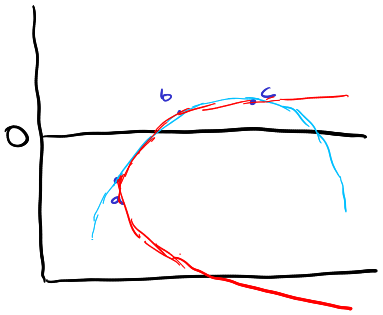
```
// b is the best estimate, c is on the other side of the root, a
   is the previous b
c ← a
// d is the signed step b will move by, e is the previous step;
   both start as the bracket width
d ← b − a;   e ← d
repeat
    if f(b) · f(c) > 0 then
        // b and c no longer bracket the root; a does
        c ← a;   d ← b − a;   e ← d
    end
    if |f(c)| < |f(b)| then
        // make b the point with the smallest |f|
        a ← b;   b ← c;   c ← a
    end
    m ← (c − b) / 2                                  // bisection step
    δ ← ε_s |b|                                      // absolute tolerance
    if |m| ≤ δ or f(b) = 0 then
        return b
    end
    if |e| < δ or |f(a)| ≤ |f(b)| then
        d ← m;   e ← m                               // bisect
    else
        s ← f(b) / f(a)
        if a = c then
            // only two distinct points:  secant step
            p ← 2ms;   q ← 1 − s
        else
            // three distinct points:  inverse quadratic
               interpolation
            q ← f(a) / f(c);   r ← f(b) / f(c)
            p ← s (2mq (q − r) − (b − a)(r − 1))
            q ← (q − 1)(r − 1)(s − 1)
        end
        if p > 0 then q ← −q
        else p ← −p
        if 2p < 3mq − |δq| and p < |eq/2| then
            // interpolated step stays in the bracket and shrinks
               it fast enough
            e ← d;   d ← p / q
        else
            d ← m;   e ← m                           // bisect
        end
    end
    a ← b
```

← bisection

← secant

← inverse quadratic

- Gauranteed to converge
- As fast near root as open methods



Fixed Point

Previously $f(x) = 0$

Fixed Point $\boxed{x = g(x)}$

repeatedly evaluate $g(x)$

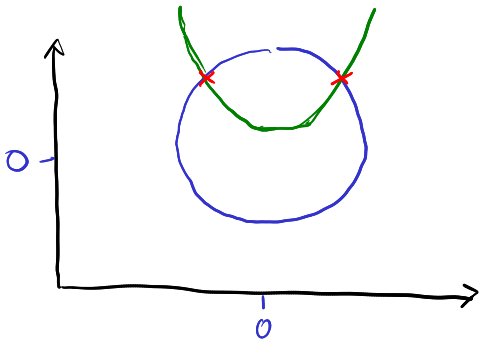
Today

Systems of nonlinear equations
Newton-Raphson for systems
Numerical differentiation

Announcements

- HW 3 due Thurs
- Exam Next Monday
  - Will release Eqn. Sheet / practice exam today / tomorrow
  - Calculator
  - Study HW, Labs, Notes

Up to now $f(x) = 0$



$x_1^2 + x_2^2 = 4$

$x_2 = x_1^2$

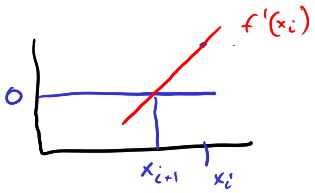
$\{f\}(\{x\}) = \{0\}$

$\vec{f}(\vec{x}) = \vec{0}$

$$\vec{f}(\vec{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ x_2 - x_1^2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Newton-Raphson for scalar:

$$f(x_i) = f(x_{i+1}) + f'(x_i)(x_i - x_{i+1})$$



vector case:

choose $\vec{x}_{i+1}$ so this is zero

first element of f →

$$f_1(\vec{x}_i) = f_1(\vec{x}_{i+1}) + \frac{\partial f_1}{\partial x_1}(x_{1,i} - x_{1,i+1}) + \frac{\partial f_1}{\partial x_2}(x_{2,i} - x_{2,i+1}) + \dots \frac{\partial f_1}{\partial x_n}(x_{n,i} - x_{n,i+1})$$

$$= \sum_{j=1}^{n} \frac{\partial f_1}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$f_2(\vec{x}_i) = \sum_{j=1}^{n} \frac{\partial f_2}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$\vdots$$

$$f_n(\vec{x}_i) = \sum_{j=1}^{n} \frac{\partial f_n}{\partial x_j}(x_{j,i} - x_{j,i+1})$$

$$\vec{f}(\vec{x}_i) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \left(\begin{bmatrix} x_{1,i} \\ \vdots \\ x_{n,i} \end{bmatrix} - \begin{bmatrix} x_{1,i+1} \\ \vdots \\ x_{n,i+1} \end{bmatrix} \right)$$

$$\vec{f}(\vec{x}_i) = J \left(\vec{x}_i - \vec{x}_{i+1}\right)$$

$\vec{f}(\vec{x}_i)$: known; J: known; $\vec{x}_i$: known; $\vec{x}_{i+1}$: unknown

$$J^{-1}\vec{f}(\vec{x}_i) = \vec{x}_i - \vec{x}_{i+1}$$

$$\boxed{\vec{x}_{i+1} = \vec{x}_i - J^{-1}\vec{f}(\vec{x}_i)}$$

(scalar)

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

want $\vec{x}_{i+1} = \vec{x}_i + \Delta\vec{x}$

$\Delta\vec{x} = \vec{x}_{i+1} - \vec{x}_i$

solve $J\Delta\vec{x} = -\vec{f}(\vec{x}_i)$

$\vec{x}_{i+1} \leftarrow \vec{x}_i + \Delta\vec{x}$

or

$\vec{x}_{i+1} \leftarrow \vec{x}_i - J \backslash \vec{f}(\vec{x}_i)$

$A \backslash b \equiv$ solution to $Ax = b$

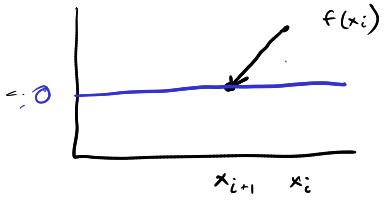
Newton-Raphson for Systems

input : function $\{f\} : \mathbb{R}^n \to \mathbb{R}^n$ and its Jacobian $[J]$ (with $J_{ij} = \partial f_i / \partial x_j$), initial guess $\{x\}_0$, tolerance ε_s, maximum iterations N

output: estimate $\{x\}_r$ of the root

$\{x\}_r \leftarrow \{x\}_0$
for $i \leftarrow 1$ **to** N **do**
  $\{x\}_{r,\text{old}} \leftarrow \{x\}_r$
  solve $[J](\{x\}_{r,\text{old}})\{\Delta x\} = -\{f(\{x\}_{r,\text{old}})\}$
  $\{x\}_r \leftarrow \{x\}_{r,\text{old}} + \{\Delta x\}$
  $\varepsilon_a \leftarrow \dfrac{\|\{x\}_r - \{x\}_{r,\text{old}}\|}{\|\{x\}_r\|} \cdot 100\%$
  if $\varepsilon_a < \varepsilon_s$ **then**
    return $\{x\}_r$
  end
end
return $\{x\}_r$ // did not converge in N iterations

(scalar)



Two Challenges

1. Convergence
2. Calculating Jacobian

Strength

"quadratic convergence" near root

Optimization

minimize $\sum_{k=1}^{n} \left(f_k(\vec{x})\right)^2$

Calculating Derivatives

Symbolic

- By hand
- Computer Algebra System (CAS)
  - e.g. Symbolic Toolbox
- Automatic Code gen

Numerical

Automatic

- Tracks exact derivative through each operation

Numerical Differentiation

Forward: $f'(x) \approx \frac{f(x+h) - f(x)}{h}$

Central: $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$ ← more accurate

Column of Jacobian → $\frac{\partial \vec{f}}{\partial x_j} \approx \frac{\vec{f}(\vec{x} + h\vec{e}_j) - \vec{f}(\vec{x})}{h}$

size of h ?

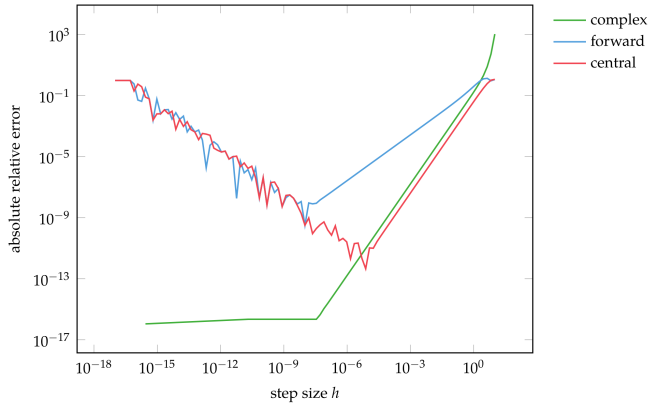
$h \approx \sqrt{\varepsilon}\, x$

↑ machine precision eps

Numerical Jacobian (forward difference)

input : function $\{f\} : \mathbb{R}^n \to \mathbb{R}^n$, point $\{x\}$, step size h
output: approximation $[J]$ of the Jacobian at $\{x\}$, $J_{ij} \approx \partial f_i / \partial x_j$

$\{f\}_0 \leftarrow \{f(\{x\})\}$
for $j \leftarrow 1$ **to** n **do**
  $\{e_j\} \leftarrow$ unit vector in the j direction // jth column of $[I]$
  $[J][:, j] \leftarrow \frac{\{f(\{x\} + h\{e_j\})\} - \{f\}_0}{h}$ // jth column of $[J]$
end
return $[J]$



Taylor Series

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)h^2}{2} + \dots$$

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \underbrace{\frac{f''(x)h}{2} + \dots}_{\text{Truncation Error}}$$

Roundoff Error

$$\frac{2\varepsilon |f(x)|}{h}$$

Best point is where these are equal

$$\frac{2\varepsilon |f(x)|}{h} = \frac{f''(x)h}{2}$$

$$\frac{4|f(x)|}{f''(x)}\varepsilon = h^2$$

$h \approx \sqrt{\varepsilon}\, x$

(right order of magnitude)

Today

Logarithmic + Exponential Growth

Reminders

- HW 3 Due Tomorrow
- Exam 1 Monday
  - Calculator, pencil
  - Be familiar w/ Eqn sheet (I will print)

Bisection requires $\log_2\left(\frac{\Delta x}{E_s}\right)$ function calls (usually c flops)

Algorithm 1

	time	times executed
Pick up end of strip	t_p	1
for $i = 1$ **to** L/l_s		
Fold one section (accordion style)	t_f	$\lceil L/l_s \rceil$
end		

runtime

$t_p + t_f \lceil L/l_s \rceil$

$t_p + t_f \frac{L}{l_s} = O(L)$

Algorithm 2

		time	times executed
1	Pick up end of strip	t_p	1
2	$l_t \leftarrow L$	0	1
3	**while** $l_t > l_s$		
4	~~Run to other end~~		
5	Pick up end and fold in half	t_f	$\log_2\left(\frac{L}{l_s}\right)$
6	$l_t \leftarrow l_t/2$	0	
7	**end**		

Iter	l_t
0	L
1	$\frac{L}{2}$
2	$\frac{L}{4}$
k	$\frac{L}{2^k}$

$l_s = l_t = \frac{L}{2^k}$

$2^k = \frac{L}{l_s}$

$k = \log_2\left(\frac{L}{l_s}\right)$

runtime:

$t_p + t_f \log_2\left(\frac{L}{l_s}\right)$

$= O(\log(L))$

Algorithm 2 (Running)

		time	times executed
1	Pick up end of strip	t_p	1
2	$l_t \leftarrow L$	0	
3	**while** $l_t > l_s$		
4	Run to other end	l_t/v	$\log\left(\frac{L}{l_s}\right)$
5	Pick up end	t_p	$\log\left(\frac{L}{l_s}\right)$
6	$l_t \leftarrow l_t/2$		
7	**end**		

runtime:

$t_p + \sum_{j=1}^{\log_2\left(\frac{L}{l_s}\right)} \left(\frac{L}{2^j}\frac{1}{v} + t_p\right)$

$= t_p\left(1 + \log_2\left(\frac{L}{l_s}\right)\right) + \sum_{j=1}^{\log_2\left(\frac{L}{l_s}\right)} \frac{L}{2^j}\frac{1}{v}$

$\leq t_p(\text{"}\quad\text{"}) + \frac{L}{v}\sum_{j=1}^{\infty} 2^{-j}$

$= t_p\left(1 + \log_2\left(\frac{L}{l_s}\right)\right) + \frac{2L}{v}$

$= O(L)$

$v = 10\,m/s$

Alg 1 and Alg 2(R) both have $O(L)$ runtime

Alg 1 → $\frac{t_f}{l_s} L$ Alg 2(R) → $\frac{2}{v} L$

$\frac{l_s}{t_f} = \frac{1/3\,m}{1\,s}$

$\frac{t_f}{l_s} = 3$ $\quad \frac{2}{v} = 0.2$

Alg 2 (R) takes ~ 15x Alg

Logarithmic Growth

$O(1)$ $O(\log(n))$ $O(n^{0.001})$ $O(n^{1/2})$ $O(n)$ $O(n^2)$ $O(n^3)$ $O(e^n)$

$\log(n)$ grows slower asymtotically than <u>any</u> polynomial n^ε $\varepsilon>0$

$f(n)=O(n)$ ↑ $f(n)\in O(n)$ →

$$O(\log(n)) \subsetneq O(n^\varepsilon) \quad \forall \varepsilon>0$$

↑ strict subset

For sets of functions $O(f(n))$ and $O(g(n))$: $O(f(n)) \subsetneq O(g(n))$ means

1. $O(g(n))$ contains every function in $O(f(n))$
2. $O(g(n))$ contains at least 1 function **NOT** in $O(f(n))$

How can we prove this

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \implies O(f(n)) \subsetneq O(g(n))$$

$$\lim_{n\to\infty} \frac{\log(n)}{n^\varepsilon} \overset{\text{L'Hopital}}{=} \lim_{n\to\infty} \frac{n^{-1}}{\varepsilon n^{\varepsilon-1}} = \lim_{n\to\infty} \frac{1}{\varepsilon n^\varepsilon} = 0$$

$$\therefore \boxed{O(\log(n)) \subsetneq O(n^\varepsilon) \quad \forall \varepsilon>0}$$

$$\log_a(n) = \frac{\log_b(n)}{\log_b(a)} \leftarrow \text{constant} \qquad \therefore \boxed{O(\log_a(n)) = O(\log_b(n)) \quad \forall a,b>0}$$

Exponential Growth

a^n, $a>1$ grows faster than <u>any</u> polynomial

$$e^n = \sum_{j=0}^{\infty} \frac{n^j}{j!} \qquad a^n = \sum_{j=0}^{\infty} \frac{\log(a)^j n^j}{j!} \geq \frac{\log(a)^{k+1} n^{k+1}}{(k+1)!}$$

$$\lim_{n\to\infty} \frac{n^k}{a^n} \leq \lim \frac{(k+1)!}{\log(a)^{k+1}} \frac{n^k}{n^{k+1}} = \lim_{n\to\infty} C \frac{1}{n} = \underline{0}$$

$$\therefore \boxed{O(n^k) \subsetneq O(a^n) \quad \forall a>1,\ k>0}$$

Let $1<a<b$

$$\boxed{O(a^n) \subsetneq O(b^n) \quad \forall \ 1<a<b}$$

Factorial Growth

$n! = n(n-1)(n-2)\cdots 1$ grows faster than any exponential

$$\lim_{n\to\infty} \frac{a^n}{n!} = \lim_{n\to\infty} \frac{\overbrace{a\cdot a\cdots a}^{N} \overbrace{a\cdots a\cdot a}^{n-N}}{\underbrace{1\cdot 2\cdots N}_{N} \underbrace{(N+1)(N+2)\cdots n}_{n-N}} \leq \lim_{n\to\infty} \frac{a^N}{N!} \left(\frac{1}{2}\right)^{n-N} = 0$$

$N = 2a$

$$\boxed{O(a^n) \subsetneq O(n!) \quad \forall a>1}$$

$$O(1) \subsetneq O(\log(n)) \subsetneq O(n) \subsetneq O(n^k) \subsetneq O(a^n) \subsetneq O(n!)$$

n	$\log_2 n$	n	n^3	2^n	time	$n!$	time
10	3.3	10	10^3	10^3	μs	3.6×10^6	ms
20	4.3	20	8×10^3	10^6	ms	2.4×10^{18}	77 yr
30	4.9	30	2.7×10^4	10^9	1 s	2.7×10^{32}	10^{15} yr
60	5.9	60	2×10^5	10^{18}	36 yr	8×10^{81}	10^{65} yr

Times assume one operation per flop at 10^9 flop/s.

$\det(A)$ with Laplace Expansion requires $n!$ flops

Cramer's $A^{-1} = \frac{\text{adj}(A)}{\det(A)}$ is extremely inefficient

Today
ODEs
Euler's

Announcements
Good Luck on Exams!
Pdf + Code

$\dot{x} \equiv \frac{dx}{dt} = ax$

$\frac{dx}{dt} - ax = 0$

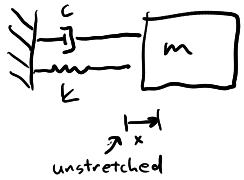
_____ equation

differential : involves derivatives
ordinary : only 1 indep. variable (t)
linear : each term is linear (no x^2, $\sin x$)
first order : only involves first derivatives
homogeneous : no "forcing" / 0 rhs

as opposed to PDE

$x = Ce^{at}$
↑ determined by boundary condition

$x = x_0 e^{at}$



$f = ma$

$-kx - c\dot{x} = m\ddot{x}$

$m\ddot{x} + c\dot{x} + kx = 0$

$\ddot{x} = \frac{d^2x}{dt^2}$

differential
ordinary
linear
second order
homogeneous

char. eqn.

$m\lambda^2 + c\lambda + k = 0$

$\lambda = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m}$

Damping

Over: λ: 2 distinct real

$x = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t}$

Critically $\lambda = \frac{-c}{2m}$ (repeated)

$x = (C_1 + C_2 t) e^{\lambda t}$

Under $\lambda = \sigma \pm i\omega$

$x = e^{\sigma t}\left(C_1 \sin(\omega t) + C_2 \cos(\omega t)\right)$

Roughly: solutions to linear ODEs are exponentials or growing or decaying oscillations (sin/cos)

Nonlinear ODEs ???

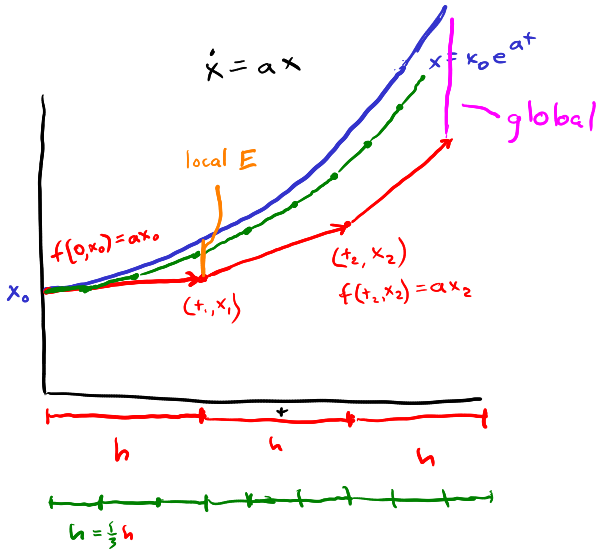
Numerical Integration

$\dot{x} = f(t, x)$
↑ dynamics

Euler's Method

$t_{i+1} = t_i + h$ ← step size

$x_{i+1} = x_i + f(t_i, x_i)\, h$



Error Analysis

Truncation (Taylor Series) — local (each step) / global (after all steps)

Roundoff (floating point sig figs)

Taylor Series

$$x_{i+1} = x_i + f(t_i, x_i)h + \frac{\partial f}{\partial t}(t_i, x_i)\frac{h^2}{2!} + \frac{\partial^2 f}{\partial t^2}(t_i, x_i)\frac{h^3}{3!} \cdots$$

Euler update

Local $E_t = \frac{\partial f}{\partial t}(t_i, x_i)\frac{h^2}{2!} + O(h^3)$

Local $E_t = O(h^3)$ as $h \to 0$

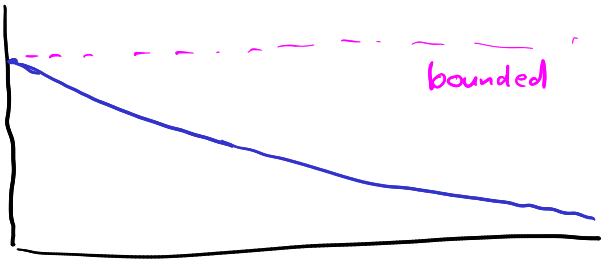
Global $E_t = O(h)$ as $h \to 0$

Stability of numerical ODE integrators

A numerical integrator is said to be unstable if errors grow exponentially for a problem with a bounded solution.

$\dot{x} = -ax \quad a > 0$

$x = x_0 e^{-at}$



Euler's $f(t_i, x_i)$

$x_{i+1} = x_i - a x_i h$

$= x_i(1 - ah)$

$x_n = x_0(1-ah)^n$

amplification factor

x_n shrinks if $|1-ah| < 1$

grows exponentially if $|1-ah| > 1$

Stability depends on 3 factors: ODE, Integration method, step size h

For $\dot{x} = -ax \quad a > 0$

- $h > \frac{2}{a}$ Euler's unstable
- $h < \frac{2}{a}$ Euler's stable

Euler's method is conditionally stable

What about systems? higher order?

$$\dot{\vec{x}} = f(t, \vec{x})$$

$$m\ddot{x} + c\dot{x} + kx = 0$$

$$v \equiv \dot{x}$$

$$\dot{v} = -\frac{c}{m}v - \frac{k}{m}x$$

$$\vec{x} = \begin{bmatrix} x \\ v \end{bmatrix}$$

state vector

$$f(t, \vec{x}) = \begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v \\ -\frac{c}{m}v - \frac{k}{m}x \end{bmatrix}$$

state derivative

Today

Runge-Kutta

Announcements

Lab 2, HW4 due Thurs

Exam Grades today hopefully

Heun's algorithm

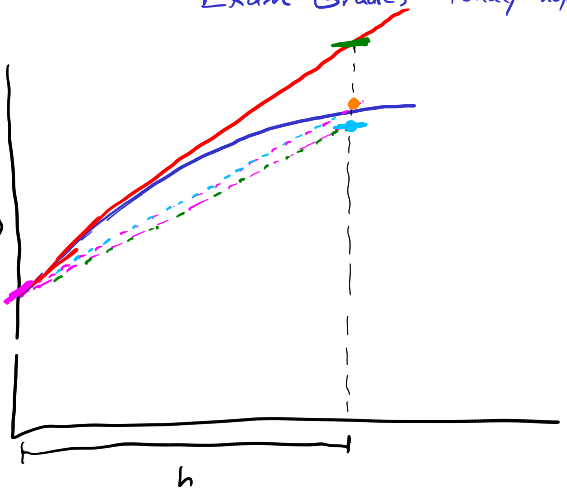
Step i:

predict: $x_{i+1}^0 \leftarrow x_i^m + f(t_i, x_i)h$

Repeat until $\left|\frac{x_{i+1}^{j+1} - x_{i+1}^j}{x_{i+1}^j}\right| < \varepsilon_s$; (m iterations over j)

correct: $x_{i+1}^j \leftarrow x_i^m + h\frac{f(t_i, x_i^m) + f(t_{i+1}, x_{i+1}^{j-1})}{2}$

$j \leftarrow j+1$



Local: $O(h^3)$ Global: $O(h^2)$

Runga Kutta

Classical RK4 "4th order Runga Kutta"

update

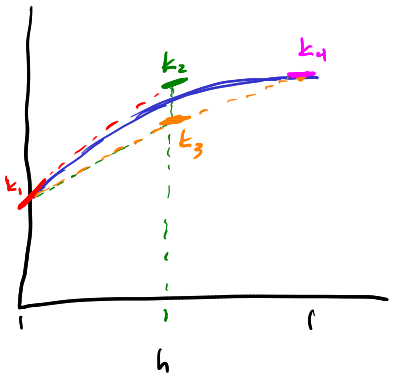
$$x_{i+1} \leftarrow x_i + \underbrace{\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)}_{\phi} h$$

$k_1 = f(t_i, x_i)$

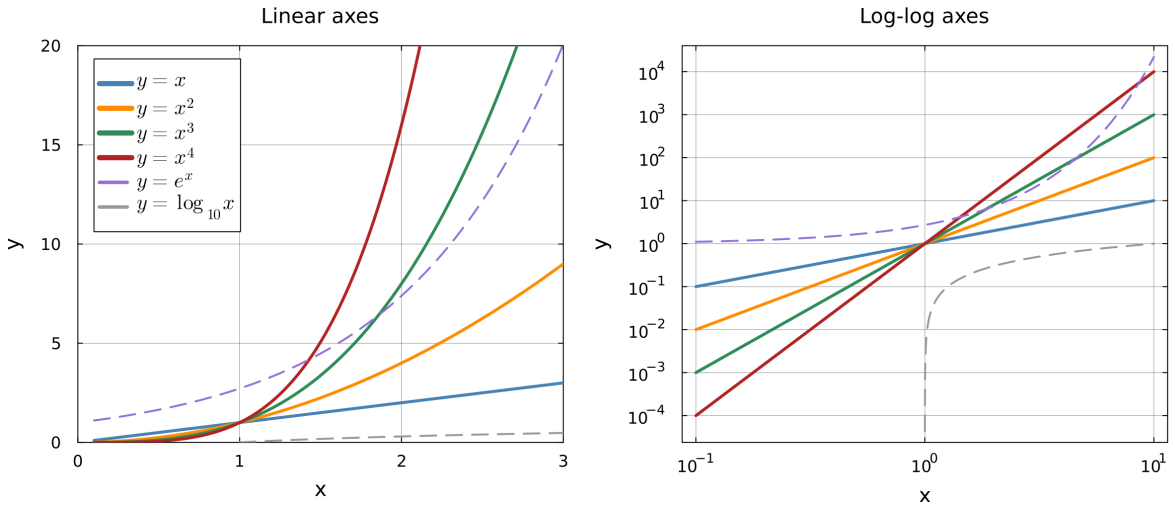
$k_2 = f(t_i + \frac{h}{2}, x_i + \frac{1}{2}k_1 h)$

$k_3 = f(t_i + \frac{h}{2}, x_i + \frac{1}{2}k_2 h)$

$k_4 = f(t_i + h, x_i + k_3 h)$



Local: $O(h^5)$ Global: $O(h^4)$



coordinates on log-log

$x' = \log(x)$ $y' = \log(y)$ $y = x^k$

$y' = \log(x^k) = k\log(x) = kx'$

$y' = kx'$

General Runge-Kutta

$$x_{i+1} \leftarrow x_i + \phi h$$

$$\phi = a_1 k_1 + a_2 k_2 + \dots a_n k_n$$

$$k_1 = f(t_i, x_i)$$

$$k_2 = f(t_i + p_1 h,\ x_i + q_{11} k_1 h)$$

$$k_3 = f(t_i + p_2 h,\ x_i + q_{21} k_1 h + q_{22} k_2 h)$$

$$\vdots$$

$$k_n = f(t_i + p_{n-1} h,\ x_i + q_{n-1,1} k_1 h + \dots q_{n-1,n-1} k_{n-1} h)$$

First order

$$x_{i+1} \leftarrow x_i + k_1 h$$

Euler's

How to find a's, p's, q's? Set equal to order-n Taylor Series

Example: Second Order

$$x_{i+1} \leftarrow x_i + (a_1 k_1 + a_2 k_2) h$$

$$f_i \equiv f(t_i, x_i) = k_1$$

$$k_2 = f(t_i + ph,\ x_i + q k_1 h)$$

$$x_{i+1} = x_i + a_1 f_i h + a_2 f(t_i + ph,\ x_i + q f_i h) h$$

$$= x_i + a_1 f_i h + a_2 \left(f_i + \frac{\partial f}{\partial t}\Big|_i ph + \frac{\partial f}{\partial x}\Big|_i q f_i h\right) h$$

Taylor Series

$$x_{i+1} \approx x_i + \frac{dx}{dt}\Big|_i h + \frac{d^2x}{dt^2}\Big|_i \frac{h^2}{2} + \dots$$

second order truncation ⇒ second order RK method

$$= x_i + f_i h + \left(\frac{\partial f}{\partial t}\Big|_i + \frac{\partial f}{\partial x}\Big|_i f_i\right) \frac{h^2}{2}$$

chain rule

$$\frac{d^2x}{dt^2}\Big|_i = \frac{df}{dt}\Big|_i = \frac{\partial f}{\partial t}\Big|_i + \frac{\partial f}{\partial x}\Big|_i \frac{dx}{dt}\Big|_i = \frac{\partial f}{\partial t}\Big|_i + \frac{\partial f}{\partial x}\Big|_i f_i$$

equal to each other

$$a_1 + a_2 = 1$$

$$p h^2 a_2 = \frac{h^2}{2} \quad \Rightarrow \quad p a_2 = \frac{1}{2}$$

$$a_2 q f_i h^2 = f_i \frac{h^2}{2} \quad \Rightarrow \quad a_2 q = \frac{1}{2}$$

$a_1 + a_2 = 1$ $\quad a_2 p = \frac{1}{2}$ $\quad a_2 q = \frac{1}{2}$

3 equations, 4 unknowns — under-determined

① $a_2 = \frac{1}{2} \Rightarrow a_1 = \frac{1}{2},\ p = q = 1$ — Heun's with no iter

② $a_2 = 1 \Rightarrow a_1 = 0,\ p = q = \frac{1}{2}$ — Midpoint

③ $a_2 = \frac{3}{4} \Rightarrow a_1 = \frac{1}{4},\ p = q = \frac{2}{3}$ — Ralstons

Today

Adaptive Runga Kutta

Stiff ODEs

Implicit Methods

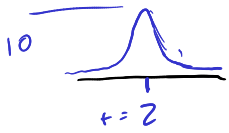
Announcements

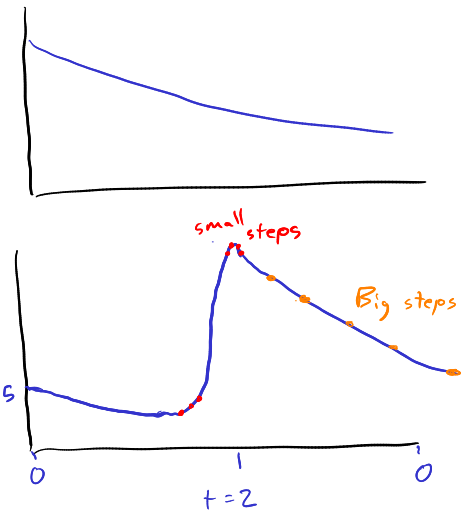
Lab 2, HW 4 due tomorrow

$\dot{x} = -0.6x \Rightarrow x = x_0 e^{-0.6t}$

$\dot{x} = 10e^{-(t-2)^2/[2(0.075)^2]} - 0.6x$

takes value of 10 at $t=2$





Option 1

Step Halving

Option 2: Adaptive Runge Kutta

```
1: function EULER_ADAPTIVE(f, [t_0, t_f], x_0, E_s)
2:     t ← [t_0],   x ← [x_0],   h ← (t_f − t_0)/100
3:     while t_end < t_f do
4:         h ← min(h, t_f − t_end)
5:         x_full ← x_end + h f(t_end, x_end)                ▷ or RK4
6:         x_mid ← x_end + (h/2) f(t_end, x_end)
7:         x_half ← x_mid + (h/2) f(t_end + h/2, x_mid)
8:         E_a ← |x_half − x_full|
9:         if E_a ≤ E_s then
10:            append t_end + h to t
11:            append x_half to x
12:        end if
13:        h ← 0.9 h sqrt(E_s/E_a)                           ▷ b/c E_a is O(h^2)
14:    end while
15:    return t, x
16: end function
```

full Euler's

2 Euler steps with h/2

```
1: function BS23_ADAPTIVE(f, [t_0, t_f], x_0, E_s)
2:     t ← [t_0],   x ← [x_0],   h ← (t_f − t_0)/100
3:     k_1 ← f(t_0, x_0)
4:     while t_end < t_f do
5:         h ← min(h, t_f − t_end)
6:         k_2 ← f(t_end + h/2, x_end + (h/2) k_1)
7:         k_3 ← f(t_end + 3h/4, x_end + (3h/4) k_2)
8:         x_3 ← x_end + h (2/9 k_1 + 1/3 k_2 + 4/9 k_3)     ▷ 3rd order
9:         k_4 ← f(t_end + h, x_3)
10:        x_2 ← x_end + h (7/24 k_1 + 1/4 k_2 + 1/3 k_3 + 1/8 k_4)   ▷ 2nd order
11:        E_a ← |x_3 − x_2|
12:        if E_a ≤ E_s then
13:            append t_end + h to t
14:            append x_3 to x
15:            k_1 ← k_4                                     ▷ slope at the new x_end
16:        end if
17:        h ← 0.9 h (E_s/E_a)^{1/3}                         ▷ b/c E_a is O(h^3)
18:    end while
19:    return t, x
20: end function
```

3 function evaluations per step

k_i for next iter

Local: $O(h^3)$

Start with this IRL

ode23

ode45 does the same thing with 4th/5th order

Stiff ODEs

A "stiff" system involves rapidly-changing and slowly-changing components



Van-der-pol Oscillator

$\ddot{x} - \mu(1-x^2)\dot{x} + x = 0$

first create system $\dot{\vec{x}} = f(t, \vec{x})$

$v \equiv \dot{x}$

$\dot{\vec{x}} = f(t, \vec{x})$

$$\begin{bmatrix} \dot{v} \\ \dot{x} \end{bmatrix} = \begin{bmatrix} \mu(1-x^2)v - x \\ v \end{bmatrix}$$

system of 1st order ODEs

$\mu = 1$: not stiff

RK4 works

BS23 works

$\mu = 1000$: stiff

RK4 diverges

BS23 takes a lot of steps

Implicit methods

help stability not accuracy

Implicit Euler Update

Explicit $x_{i+1} \leftarrow x_i + f(t_i, x_i)\,h$

Implicit $x_{i+1} \leftarrow x_i + f(t_i+h, x_{i+1})\,h$

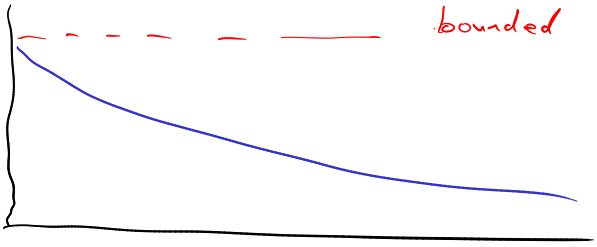
How do we find x_{i+1}?

Solve (possibly nonlinear equation) $x_{i+1} - x_i - f(t_{i+1}, x_{i+1})\,h = 0$

Use any nonlinear solver

Example

$\dot{x} = -ax \qquad a > 0$



unstable if errors grow exponentially for a bounded solution

Explicit Euler was conditionally stable

Implicit: $x_{i+1} = x_i - a x_{i+1} h$

$x_{i+1}(1+ah) = x_i$

$x_{i+1} = \frac{x_i}{1+ah}$ ← shrinks regardless of h

Implicit Euler is unconditionally stable

Implicit Euler on VDP equations

$$v_{i+1} = v_i + \mu(1 - x_{i+1}^2)\,v_{i+1} - x_{i+1}$$
$$x_{i+1} = x_i + v_{i+1}$$

unknowns

nonlinear system of equations

Newton-Raphson for systems

IRL if you have a stiff system consider this

ode23s in Matlab

↑ stiff

Today

Boundary Value Problems

Shooting Method

Finite Difference

Announcements

HW5 due Thursday (no late penalty)

So far: initial value problems

$$\vec{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \qquad \dot{\vec{x}} = f(t, \vec{x}) \qquad \begin{matrix} x(0) = a \\ \dot{x}(0) = b \end{matrix}$$ ← initial conditions

Boundary-value problems

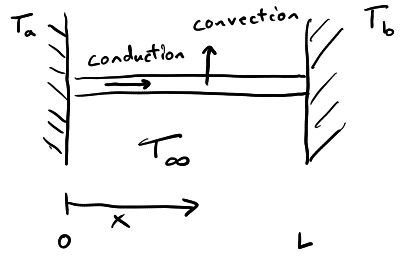
Example: $x(0) = a$ (initial pos.) $\qquad x(T) = b$ (final pos.)

$\dot{x}(0), \dot{x}(T)$ unknown

---

x known: Dirichlet B.C. $\qquad$ $\dot{x}$ known: Neumann B.C.

Example: heated rod



$$\frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma'(T_\infty - T) = 0 \qquad \text{(radiation)}$$

$$T(0) = T_a \qquad T(L) = T_b$$

Solve analytically

$$\vec{y} = \begin{bmatrix} T \\ \frac{dT}{dx} \end{bmatrix} \qquad \frac{d\vec{y}}{dx} = f(t, \vec{y})$$

$$\vec{y}(0) = \begin{bmatrix} T(0) = T_a \\ \frac{dT}{dx}(0) \end{bmatrix} \qquad \text{ode45}(f, [0, L], \vec{y}(0))$$

Start with $\frac{T_b - T_a}{L}$, refine

Shooting Method

Start with known IC's, guess unknown ICs

Adjust unknown IC's and "shoot" until we satisfy all BCs

How do we shoot in a smart way?

Let $z \equiv \frac{dT}{dx}$ $\qquad z(0)$ is unknown IC

define $g(T(0), z(0))$ integrates (using RK4/ode45) and returns $T(L)$

$g(T_a, z(0)) = T_b \Rightarrow 0 = g(T_a, z(0)) - T_b$ $\qquad f(z(0)) = 0$

$z(0)$: unknown

nonlinear scalar equation

solve for $z(0)$ using _____

↑ your favorite nonlinear solver

Newton Raphson, Incremental Search, Secant Bisection

can diverge for complex dynamics

Finite Difference Method

1) Discretize problem into n Δx segments
2) Replace derivatives with finite difference (central finite difference)
3) solve resulting equations

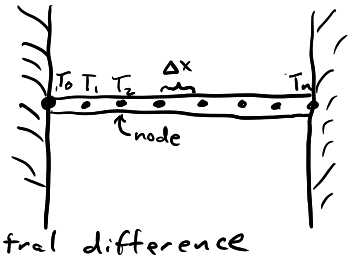
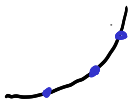


Fig 21.5: central difference

First Derivative	Error
$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h}$	$O(h^2)$
$f'(x_i) = \frac{-f(x_{i+2}) + 8f(x_{i+1}) - 8f(x_{i-1}) + f(x_{i-2})}{12h}$	$O(h^4)$
Second Derivative	
$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1})}{h^2}$	$O(h^2)$
$f''(x_i) = \frac{-f(x_{i+2}) + 16f(x_{i+1}) - 30f(x_i) + 16f(x_{i-1}) - f(x_{i-2})}{12h^2}$	$O(h^4)$
Third Derivative	
$f'''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + 2f(x_{i-1}) - f(x_{i-2})}{2h^3}$	$O(h^2)$
$f'''(x_i) = \frac{-f(x_{i+3}) + 8f(x_{i+2}) - 13f(x_{i+1}) + 13f(x_{i-1}) - 8f(x_{i-2}) + f(x_{i-3})}{8h^3}$	$O(h^4)$
Fourth Derivative	
$f''''(x_i) = \frac{f(x_{i+2}) - 4f(x_{i+1}) + 6f(x_i) - 4f(x_{i-1}) + f(x_{i-2})}{h^4}$	$O(h^2)$
$f''''(x_i) = \frac{-f(x_{i+3}) + 12f(x_{i+2}) - 39f(x_{i+1}) + 56f(x_i) - 39f(x_{i-1}) + 12f(x_{i-2}) - f(x_{i-3})}{6h^4}$	$O(h^4)$

$$\frac{d^2T}{dx^2} \approx \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2}$$



$$\frac{d^2T}{dx^2} + h'(T_\infty - T) = 0$$

$$\frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} + h'(T_\infty - T_i) = 0$$

$$-T_{i-1} + (2 + h'\Delta x^2)T_i - T_{i+1} = h'\Delta x^2 T_\infty$$

$T_0 = T_a \qquad T_n = T_b$

case: $L = 10m \qquad h' = 0.05\ m^{-2}$

$T_\infty = 200K \qquad T_a = 300K, \quad T_b = 400K$

$i = 1$: $-T_a + (2 + h'\Delta x^2)T_1 - T_2 = h'\Delta x^2 T_\infty + T_a$

$i = 2$: $-T_1 + (2 + h'\Delta x^2)T_2 - T_3 = h'\Delta x^2 T_\infty$

$\vdots$

$i = n-1$: $-T_{n-2} + (2 + h'\Delta x^2)T_{n-1} - T_n = h'\Delta x^2 T_\infty + T_b$

plug in numbers and put into matrix form:

$2 + h'\Delta x^2 = 2.2, \quad h'\Delta x^2 T_\infty = 40, \quad n = 5$

$$\begin{bmatrix} 2.2 & -1 & 0 & 0 \\ -1 & 2.2 & 0 & 0 \\ 0 & -1 & 2.2 & -1 \\ 0 & 0 & -1 & 2.2 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 40 + 300 \\ 40 \\ 40 \\ 40 + 400 \end{bmatrix}$$

solve with linear solver.

Since this is tridiagonal, you can use an optimized $O(n)$ solver. Matlab or Julia's `x = A\b` will do this automatically.